3

The graph CF recommendation pipeline

This section provides the required technical background regarding graph-based collaborative filtering (CF). By considering the main solutions depicted in the literature, we recognize some recurrent strategy patterns that help us designing a general graph CF recommendation pipeline. In the following, we focus on each stage of the pipeline, namely, the graph input data, the processing of the graph structure, the propagation procedures, and the final recommendation task (Figure 3.1). Finally, we formally define a model taxonomy over the selected eleven recommendation models.

3.1 Graph input

The graph input data, in the minimum settings, requires one piece of information: the bipartite user-item interaction graph, formally represented by an adjacency matrix. Optionally, users' and items' nodes may come with (learnable) features, represented as node embeddings. In the following, we formalize both components.

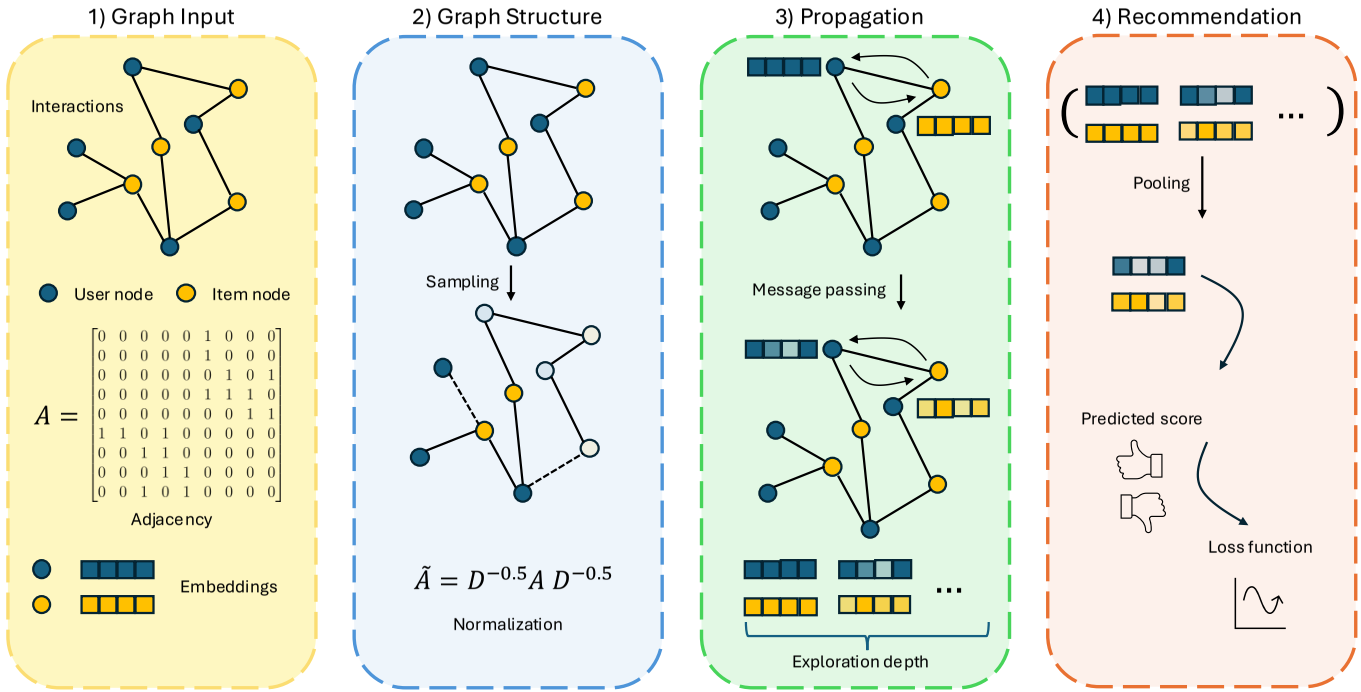


Figure 3.1: Overall graph CF pipeline, consisting of four main steps: 1) graph input, where we consider the user-item graph structure with optional node embeddings; 2) graph structure, where we perform optional modifications to the graph structure to improve its informativeness; 3) propagation, where we optionally propagate messages at multiple hops through the message passing algorithm; 4) recommendation, where we get a compact representation of users' and items' embeddings and exploit them for the final task.

3.1.1 Interactions

Let $\mathcal{U}$ and $\mathcal{I}$ be the sets of users and items, respectively, with $|\mathcal{U}| = U$ and $|\mathcal{I}| = I$ as the total number of users and items in the catalog. Then, let $\mathbf{R} \in \mathbb{R}^{U \times I}$ be the user-item interaction matrix where, assuming an implicit feedback setting, $R_{ui} = 1$ if user $u \in \mathcal{U}$ interacted with item $i \in \mathcal{I}$, 0 otherwise. We build the adjacency matrix $\mathbf{A} \in \mathbb{R}^{(U+I)\times(U+I)}$ which represents the bi-directional connections among users and items according to $\mathbf{R}$:

$$\mathbf{A} = \begin{bmatrix} 0 & \mathbf{R} \\ \mathbf{R}^\top & 0 \end{bmatrix}. \tag{3.1}$$

On the above premises, we formally define the user-item bipartite and undirected graph $\mathcal{G} = \{\mathcal{U} \cup \mathcal{I}, \mathbf{A}\}$. We recall that, in a bipartite graph, we see connections existing only among nodes from the two partitions (i.e., users with items and vice versa). However, there exist other types of connections which are not explicitly encoded, such as the user-user connections and the item-item ones. To obtain those, let us first partition the adjacency matrix as follows:

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}^{\mathcal{U}\mathcal{U}} & \mathbf{A}^{\mathcal{U}\mathcal{I}} \\ \mathbf{A}^{\mathcal{I}\mathcal{U}} & \mathbf{A}^{\mathcal{I}\mathcal{I}} \end{bmatrix}, \tag{3.2}$$

where, in the original user-item graph, $\mathbf{A}^{\mathcal{U}\mathcal{U}} = 0$ and $\mathbf{A}^{\mathcal{I}\mathcal{I}} = 0$. Nevertheless, if we consider the user- and item-*projected* graphs as $\mathcal{G}_\mathcal{U} = \{\mathcal{U}, \mathbf{A}^{\mathcal{U}\mathcal{U}}\}$ and $\mathcal{G}_\mathcal{I} = \{\mathcal{I}, \mathbf{A}^{\mathcal{I}\mathcal{I}}\}$, we can explicitly derive those connections. In this respect, let $\mathbf{R}^{\mathcal{U}\mathcal{U}}$ and $\mathbf{R}^{\mathcal{I}\mathcal{I}}$ be the user-user and item-item interaction matrices:

$$\mathbf{R}^{\mathcal{U}\mathcal{U}} = \mathbf{R} \cdot \mathbf{R}^\top, \qquad \mathbf{R}^{\mathcal{I}\mathcal{I}} = \mathbf{R}^\top \cdot \mathbf{R}, \tag{3.3}$$

which store the co-occurrences among users and items, respectively. Trivially, the corresponding adjacency matrices $\mathbf{A}^{\mathcal{U}\mathcal{U}}$ and $\mathbf{A}^{\mathcal{I}\mathcal{I}}$ are obtained as:

$$\mathbf{A}^{\mathcal{U}\mathcal{U}} = \mathbf{R}^{\mathcal{U}\mathcal{U}}, \qquad \mathbf{A}^{\mathcal{I}\mathcal{I}} = \mathbf{R}^{\mathcal{I}\mathcal{I}}. \tag{3.4}$$

3.1.2 Node representation

We optionally represent each user and item node with its own features. In this respect, the tendency is to leverage latent factor-based representations, which are carefully initialized and trained end-to-end in

the downstream task. Specifically, let $\mathbf{E}_u \in \mathbb{R}^F$ and $\mathbf{E}_i \in \mathbb{R}^F$ be the embedding representations for user u and item i, respectively, where F is their vector dimension (i.e., $F << U, I$). Given the iterative nature of several approaches (i.e., message passing, see later), we may need to update these node embeddings for multiple hops. Thus, we will use the notation $\mathbf{E}^{(l)}$ to denote the updated representation of the node latent factor $\mathbf{E}$ at hop l (i.e., $0 \leq l \leq L$), with $\mathbf{E}^{(0)} = \mathbf{E}$ as the initialization.

While the most recurrent strategy for node representation is the one described above, it is worth underlining other important solutions from the literature. Indeed, approaches such as (Wang *et al.*, 2020) disentangle node embeddings into intents; that is, each embedding portion refers to a specific aspect (intent) underlying the user-item interaction. Then, other solutions (Cai *et al.*, 2023; Peng *et al.*, 2022b; Shen *et al.*, 2021) decompose the interaction matrix $\mathbf{R}$ through, for example, singular value decomposition, to access the intrinsic properties of the user-item interaction structure and embed it into the node representation.

3.2 Graph structure

Noteworthy, we may use some tailored procedures to modify the graph input structure so that the information conveyed by the recorded interactions is more useful to the downstream recommendation task. Specifically, the literature indicates two main strategies, namely, adjacency normalization and graph sampling. In the following, we deepen into both techniques.

3.2.1 Normalization

To limit the over-penalization of nodes with very few interactions in the user-item graph, a common approach is to normalize the adjacency matrix through the symmetric adjacency normalization. Let $\mathcal{N}_u = \{i \in \mathcal{I} \mid R_{ui} = 1\}$ and $\mathcal{N}_i = \{u \in \mathcal{U} \mid R_{ui} = 1\}$ be the set of one-hop neighbor nodes of u and i, respectively, with $|\mathcal{N}_u|$ and $|\mathcal{N}_i|$ their cardinalities (i.e., the number of interactions for user u and item i). Then, let $\mathbf{D} \in \mathbb{R}^{(U+I)\times(U+I)}$ be the diagonal degree matrix, where $D_{v,v} = \sum R_{v,*} = |\mathcal{N}_v|$ is the degree of the generic graph node v. Similarly

to what we did for the adjacency matrix (Equation 3.2), we can partition it as:

$$\mathbf{D} = \begin{bmatrix} \mathbf{D}^{\mathcal{U}\mathcal{U}} & \mathbf{D}^{\mathcal{U}\mathcal{I}} \\ \mathbf{D}^{\mathcal{I}\mathcal{U}} & \mathbf{D}^{\mathcal{I}\mathcal{I}} \end{bmatrix}. \tag{3.5}$$

Then, the symmetric normalization of the adjacency matrix is calculated as:

$$\tilde{\mathbf{A}} = \mathbf{D}^{-\frac{1}{2}}\mathbf{A}\mathbf{D}^{-\frac{1}{2}}, \tag{3.6}$$

where the generic entry is as follows:

$$\tilde{A}_{u,i} = \frac{1}{\sqrt{|\mathcal{N}_u|}\sqrt{|\mathcal{N}_i|}}. \tag{3.7}$$

This type of normalization prevents nodes with low degree (i.e., few interactions) from being penalized over nodes with high degree (i.e., numerous interactions). Various approaches in the literature (He *et al.*, 2020; Peng *et al.*, 2022b; Wang *et al.*, 2019; Yang *et al.*, 2023; Yu *et al.*, 2024) adopt the symmetric normalization, with some variants that apply it during the learning of the adjacency matrix (Wang *et al.*, 2020) and that reformulate it to tackle some issues such as the over-smoothing effect (Mao *et al.*, 2021).

3.2.2 Sampling

Graph structure sampling aims at exploring specific paths in the graph structure and/or regularizing the training of the models by encouraging the agreement between different views of the same user-item graph.

Random walk (i.e., RW) is a stochastic process on graphs that simulates the path that, starting from seed nodes, traverses the neighbors with a uniform probability up to a given depth. As for training regularization (Cai *et al.*, 2023; Wang *et al.*, 2019; Wu *et al.*, 2021), there are approaches (Shu *et al.*, 2022) which follow a similar rationale to dropout in machine learning, such as node, edge, and message dropout. Node and edge dropout randomly mask nodes and edges in the user-item graph, respectively, with all connected edges and nodes. Concretely, node dropout (i.e., ND) masks rows and/or columns of the adjacency matrix, while edge dropout (i.e., ED) masks specific entries in the adjacency matrix. On the other hand, message dropout (i.e., MD)

deals with the application of dropout to propagated messages during the message passing schema. The concept will be clearer in the next section, where we formally introduce the message passing procedure.

3.3 Propagation

To effectively exploit the user-item graph structure, a popular strategy is to propagate users' and items' node information across the graph topology. While this procedure is commonly known as message passing, each recommendation model may perform it in a different manner, or even replace it with ad-hoc proxies. Additionally, when the message passing occurs, it is interesting to understand up to which depth we want to make the message passing run, as different strategies may heavily impact the final performance. In the next paragraphs, we will define these outlined aspects in the graph CF pipeline.

3.3.1 Message passing

The message passing algorithm is re-proposed in recommendation to propagate messages coming from the neighborhood of each node at multiple iterative hops, thus effectively distilling the collaborative signal (Wang *et al.*, 2019). Let $v \in \mathcal{U} \cup \mathcal{I}$ be either a user or item node. The most general formulation for the message passing on node v is:

$$\mathbf{E}_v^{(l)} = \text{Update}^{(l)}(\mathbf{E}_v^{(l-1)}, \text{Aggregate}^{(l)}(\{\mathbf{E}_w^{(l-1)} \mid w \in \mathcal{N}_v\})), \tag{3.8}$$

where $\text{Update}(\cdot)$ and $\text{Aggregate}(\cdot, \cdot)$ are the update and the aggregate functions, respectively. The aggregate function collects the last representations of nodes from the ego neighbors to obtain the messages; when the ego node is a *user*, the messages are aggregated from the directly-connected *items* (those which have been interacted by the user). After that, the update function generates the new version of the ego node representation through the collected messages, optionally combining it with the ego node itself.

While message passing is the standard solution observed in graph CF, we highlight that more recent models provide proxy versions of it (Mao *et al.*, 2021), or avoid using it (e.g., singular value decomposition

as in (Peng *et al.*, 2022b; Shen *et al.*, 2021)). The reasons for these choices are to be found in the propagation depth, an aspect that we focus on in the following section.

3.3.2 Exploration depth

Another important aspect is the *depth* of exploration when we perform the message passing algorithm. Indeed, each chosen depth may have a different meaning when it comes to a bipartite graph (as in our case). For instance, a reworking of Equation 3.8 on the user side for $l \in \{2, 3\}$ allows *same*- and *different*-type node representation emerge (Anelli *et al.*, 2023a; Anelli *et al.*, 2022b). For simplicity, we omit the Update$(\cdot)$ function and simply aggregate embeddings from neighbor nodes:

$$
\begin{aligned}
\textbf{\textit{Same}} \begin{cases} \underbrace{\mathbf{E}_u^{(2)}}_{\textbf{(user)}} \end{cases} &= \text{Aggregate}^{(2)}(\{\text{Aggregate}^{(1)}(\{\underbrace{\mathbf{E}_{u'}^{(0)}}_{\textbf{user}} \mid u' \in \mathcal{N}_i\}) \mid i \in \mathcal{N}_u\}), \\
\textbf{\textit{Diff.}} \begin{cases} \underbrace{\mathbf{E}_u^{(3)}}_{\textbf{(user)}} \end{cases} &= \text{Aggregate}^{(3)}(\{\cdots \text{Aggregate}^{(1)}(\{\underbrace{\mathbf{E}_{i'}^{(0)}}_{\textbf{item}} \mid i' \in \mathcal{N}_{u'}\}) \cdots \mid i \in \mathcal{N}_u\}).
\end{aligned}
\tag{3.9}
$$

To better clarify the extent of Equation 3.9, after an **even** and an **odd** number of explored hops in the user-item bipartite graph, the message passing works by leveraging *same*- and *different*-type node connections (i.e., U-U/I-I and U-I/I-U as evident from Equation 3.9). While the existing literature does not always consider the two scenarios as distinct, it becomes fundamental to assess the influence of different node-node connections explored during the message passing. Formally, we may count the explored hops as follows: $\mathbf{E}^{(2l)}, \forall l \in \{1, 2, \ldots, \frac{L}{2}\}$ as obtained through l **same-**type node connections (denoted as *same-l*), and $\mathbf{E}^{(2l-1)}, \forall l \in \{1, 2, \ldots, \frac{L}{2}\}$ as obtained through l **different-**type node connections (denoted as *different-l*).

On another level, exploration depth has been shown to negatively affect performance for GNNs due to the over-smoothing effect (Chen *et al.*, 2020a; Zhao and Akoglu, 2020). Particularly, it is commonly observed that performing message passing for a high number of hops may drastically decrease the performance of the GNN, as node embeddings

tend to become increasingly similar in the latent space, thus losing informativeness. The same effect occurs in recommendation, where it has been empirically demonstrated that after $l = 3$ explorations, performance generally starts to worsen, e.g., (He *et al.*, 2020; Wang *et al.*, 2019). As this appears to be counteractive to the idea of exploring larger portions of the user-item interaction graph to effectively distill the collaborative signal, some approaches have begun proposing novel techniques that provide proxies to the traditional message passing (Mao *et al.*, 2021); the idea is to modify the usual procedure to propagate messages at higher hops and still avoid over-smoothing.

3.4 Recommendation

For the final stage of the pipeline, we use the updated representations of the users' and items' nodes to predict the user-item interaction score and (optionally) calculate the loss function of the overall framework. Before the score prediction, in the message passing settings, we also provide a summarized version of the node embeddings coming from each propagation step. In the following, we explore these two aspects.

3.4.1 Layer combination

The final stage in the message passing algorithm is the combination of all node embedding representations obtained in each hop l. Let $\mathbf{E}^{(0)} = \mathbf{E}$ be the node representation for the hop 0, which corresponds to the initial embedding representation of the node. Then, the layer combination is obtained as:

$$\hat{\mathbf{E}} = \text{Combine}(\{\mathbf{E}^{(0)}, \mathbf{E}^{(1)}, \ldots, \mathbf{E}^{(l)}, \ldots, \mathbf{E}^{(L)}\}), \tag{3.10}$$

where Combine$(\cdot)$ can be different operations. Possible solutions include:

- element-wise addition ($\sum_{0 \leq l \leq L} \mathbf{E}^{(l)}$)
- weighted element-wise addition ($\sum_{0 \leq l \leq L} w_l \mathbf{E}^{(l)}$)
- mean ($\frac{1}{L} \sum_{0 \leq l \leq L} \mathbf{E}^{(l)}$)
- concatenation ($\{\mathbf{E}^{(0)} \,||\, \mathbf{E}^{(1)} \,||\, \ldots \,||\, \mathbf{E}^{(l)} \,\ldots\, ||\, \mathbf{E}^{(L)}\}$)
- taking the last obtained representation ($\mathbf{E}^{(L)}$).

3.4.2 Loss

In the case of trainable recommendation models, the final node embeddings are commonly combined through the dot product to predict the interaction score, as observed in traditional approaches for collaborative filtering. Thus, most of the considered models optimize a loss function which, in many cases, adopts the Bayesian personalized ranking (BPR) (Rendle *et al.*, 2009) loss with additional and ad-hoc components.

Let $\mathcal{T} = \{(u, i, j) \mid i \in \mathcal{N}_u \wedge j \in \mathcal{I} \setminus \mathcal{N}_u\}$ be a set of triples, where $\mathcal{N}_u$ is the set of all positive items for user u (i.e., its one-hop neighbor nodes) and $\mathcal{I} \setminus \mathcal{N}_u$ is the set of all negative items for user u. BPR seeks to maximize the posterior probability that each user u prefers a positive item i over a negative item j:

$$\begin{aligned} \mathcal{L}_{\text{BPR}} = &- \sum_{(u,i,j)\sim\mathcal{T}} \ln \text{Sigmoid}(\hat{R}_{ui} - \hat{R}_{uj}) = \\ &- \sum_{(u,i,j)\sim\mathcal{T}} \ln \text{Sigmoid}(\hat{\mathbf{E}}_u^\top \hat{\mathbf{E}}_i - \hat{\mathbf{E}}_u^\top \hat{\mathbf{E}}_j), \end{aligned} \tag{3.11}$$

where $\text{Sigmoid}(\cdot)$ is the sigmoid function, and $\hat{R}_{ui}, \hat{R}_{uj}$ are the predicted scores for the pairs of user u with the positive item i, and user u with the negative item j. Starting from the BPR basis, some works propose to include additional loss components to reach certain constraints and objectives. We will deepen into them in the last part of this section, where we provide a formal model taxonomy on graph CF based upon the depicted pipeline.

3.5 Model taxonomy

Based upon the graph CF recommendation pipeline described above, in conclusion, we report the main technical aspects regarding eleven state-of-the-art graph CF models that we selected for this monograph, namely, NGCF (Wang *et al.*, 2019), DGCF (Wang *et al.*, 2020), LightGCN (He *et al.*, 2020), SGL (Wu *et al.*, 2021), UltraGCN (Mao *et al.*, 2021), GFCF (Shen *et al.*, 2021), SVD-GCN (Peng *et al.*, 2022b), SimGCL (Yu *et al.*, 2022), LightGCL (Cai *et al.*, 2023), GraphAU (Yang *et al.*, 2023), and XSimGCL (Yu *et al.*, 2024). Indeed, it allows us to derive

a formal model taxonomy representing graph CF, and we depict it in Table 3.1.

Table 3.1: Model taxonomy as observed on the selected seven representative graph CF approaches from the recent literature. As described in the graph CF pipeline, the taxonomy accounts for four main aspects: 1) graph input, 2) graph structure, 3) propagation, and 4) recommendation. Models follow the chronological order.

Models	(1) Graph Input		(2) Graph Structure		(3) Propagation		(4) Recommendation	
	*Interactions**	*Node Repr.*	*Norm.*	*Sampling***	*Message Passing*	*Exploration Depth*	*Pooling*	*Loss****
NGCF	U-I	latent fact.	✓	MD/ND	✓	3	concat	BPR
DGCF	U-I	intents	✓	✗	✓	1	sum	BPR/IDL
LightGCN	U-I	latent fact.	✓	✗	✓	3	w-sum	BPR
SGL	U-I	latent fact.	✗	ND/ED/RW	✓	3	w-sum	BPR/SSL
UltraGCN	U-I/I-I	latent fact.	✓	✗	✓	$+\infty$	✗	BPR/CL/IL
GFCF	U-I/I-I	decomp.	✗	✗	✗	1	✗	✗
SVD-GCN	U-I/U-U/I-I	decomp.	✓	✗	✗	1	✗	BPR/UL/IL
SimGCL	U-I	latent fact.	✓	✗	✓	3	avg	BPR/CL/UNL
LightGCL	U-I	latent fact./decomp.	✓	ND/ED	✓	2	sum	HL/CL
GraphAU	U-I	latent fact.	✓	✗	✓	4	w-sum	AL/UNL
XSimGCL	U-I	latent fact.	✓	✗	✓	5	w-sum	BPR/CL

**U-I: user-item, I-I: item-item, U-U: user-user*

***MD: message dropout, ND: node dropout, ED: edge dropout, RW: random walk*

****BPR: Bayesian personalized ranking, IDL: independence loss, SSL: self-supervised loss*
CL: constraint loss, UL: user loss, IL: item loss, UNL: uniform loss, HL: hinge loss, AL: alignment loss

3.5.1 NGCF

The authors from (Wang *et al.*, 2019) propose neural graph collaborative filtering (NGCF). At each hop, the model aggregates neighborhood messages and the inter-dependencies among the ego and the neighborhood nodes. The message passing is formalized as:

$$\mathbf{E}_u^{(l)} = \text{LeakyReLU}\left(\sum_{i \in \mathcal{N}_u}\left(\mathbf{W}_{\text{neigh}}^{(l)}\tilde{A}_{ui}\mathbf{E}_i^{(l-1)} + \mathbf{W}_{\text{inter}}^{(l)}\left(\mathbf{E}_i^{(l-1)} \odot \mathbf{E}_u^{(l-1)}\right) + \mathbf{b}^{(l)}\right)\right), \tag{3.12}$$

where LeakyReLU is the activation function, $\mathbf{W}_{\text{neigh}}^{(l)} \in \mathbb{R}^{F_{l-1} \times F_l}$ and $\mathbf{W}_{\text{inter}}^{(l)} \in \mathbb{R}^{F_{l-1} \times F_l}$ are the neighborhood and inter-dependencies weight matrices, respectively, $\mathbf{b}^{(l)} \in \mathbb{R}^{F_l}$ is the bias term, while $\odot$ is the Hadamard product.

3.5.2 DGCF

Wang *et al.*, 2020 design a message passing that calculates the importance of neighborhood nodes for ego nodes by disentangling the intents underlying each user-item interaction. Therfore, they propose the following formulation:

$$\mathbf{E}_u^{(l)}[t] = \sum_{i \in \mathcal{N}_u[t]} \frac{A_{ui}[t]}{\sqrt{|\mathcal{N}_u[t]|}\sqrt{|\mathcal{N}_i[t]|}}\mathbf{E}_i^{(l-1)}[t], \tag{3.13}$$

where the operator $[t]$ indicates we are considering embeddings, matrices, and sets with reference to the t-*th* intent, having $1 \leq t \leq T$. Then, assuming the chosen intents should be independent, the authors also introduce an independence loss (i.e., IDL) formulated as:

$$\mathcal{L}_{\text{IDL}} = \sum_{t=1}^{T}\sum_{t'=t+1}^{T} \text{dCor}(\hat{\mathbf{E}}[t], \hat{\mathbf{E}}[t']), \tag{3.14}$$

where dCor() is a distance correlation function calculated among all pairs of intents in the setting.

3.5.3 LightGCN

He *et al.*, 2020 propose a light graph convolutional network, dubbed as LightGCN, to simplify the message passing algorithm from NGCF.

The idea is to drop feature transformations (i.e., the weight matrices and biases) and the non-linearity applied after the message aggregation. The final formulation is:

$$\mathbf{E}_u^{(l)} = \sum_{i \in \mathcal{N}_u} \tilde{A}_{ui} \mathbf{E}_i^{(l-1)}. \tag{3.15}$$

3.5.4 SGL

Self-supervised graph learning (Wu *et al.*, 2021) (SGL) brings self-supervised (Huang *et al.*, 2022) and contrastive (Khosla *et al.*, 2020) learning to graph CF. It uses the same message passing formulation as LightGCN, but learning different views of nodes; these are obtained through sampling strategies on the adjacency matrix (i.e., ND, ED, and RW). Concretely, we have:

$$\mathbf{E}_u^{(l)}[t_1] = \sum_{i \in \mathcal{N}_u} \tilde{A}_{ui}[t_1] \mathbf{E}_i^{(l-1)}[t_2], \quad \mathbf{E}_u^{(l)}[t_2] = \sum_{i \in \mathcal{N}_u} \tilde{A}_{ui}[t_2] \mathbf{E}_i^{(l-1)}[t_2], \tag{3.16}$$

where, with a notation abuse, we re-use the same operator [t] from DGCF to indicate the two different views for the ego node, obtained through two sampling strategies. Then, the idea is to encourage the similarity between the two views of the same node, and the dissimilarity between the two views of different nodes:

$$\mathcal{L}_{\text{SSL}} = \sum_{u \in \mathcal{U}} -\ln \frac{\exp(\text{Cosine}(\hat{\mathbf{E}}_u[t_1], \hat{\mathbf{E}}_u[t_2]))}{\sum_{v \in \mathcal{U} \setminus \{u\}} \exp(\text{Cosine}(\hat{\mathbf{E}}_u[t_1], \hat{\mathbf{E}}_v[t_2]))}, \tag{3.17}$$

where the self-supervised loss function (i.e., SSL) is only shown on the user side, but the symmetric holds for the item side, while Cosine$(\cdot)$ is the cosine similarity. This formulation of the $\mathcal{L}_{\text{SSL}}$ is better known as the InfoNCE loss (Gutmann and Hyvärinen, 2010).

3.5.5 UltraGCN

Ultra simplification of graph convolutional network (Mao *et al.*, 2021) (UltraGCN) addresses some crucial issues in graph CF. Specifically, the authors propose a novel message passing algorithm that works as a proxy to the infinite-layer propagation through a single (simplified) hop.

Moreover, the adjacency matrix is normalized through a novel version of the symmetric normalization accounting for the asymmetric weighting of connected nodes in U-U and I-I connections. Thus, the single-hop message passing formulation is:

$$\hat{\mathbf{E}}_u = \sum_{i \in \mathcal{N}_u} \tilde{A}'_{ui} \mathbf{E}_i, \quad \hat{\mathbf{E}}_i = \sum_{u \in \mathcal{N}_i^{(1)}} \tilde{A}'_{iu} \, \mathbf{E}_u, \tag{3.18}$$

where we do not use any superscript as the message passing occurs in one iteration only, and $\tilde{\mathbf{A}}'$ is the novel version of the symmetric normalization as described above. Then, two loss components are introduced to tackle the over-smoothing effect and learn from the usually-unexplored type of node relationships such as item-item. The former is called constraint loss (i.e., CL), while the latter is called item loss (i.e., IL). In detail, we have:

$$\begin{aligned} \mathcal{L}_{\text{CL}} = &- \sum_{(u,i) \sim \mathcal{T}_+} t_+ \ln(\text{Sigmoid}(\hat{\mathbf{E}}_u^\top \hat{\mathbf{E}}_i)) + \\ &- \sum_{(u,j) \sim \mathcal{T}_-} t_- \ln(\text{Sigmoid}(-\hat{\mathbf{E}}_u^\top \hat{\mathbf{E}}_j)), \end{aligned} \tag{3.19}$$

where $\mathcal{T}_+ = \{(u,i) \mid R_{ui} = 1\}$ and $\mathcal{T}_- = \{(u,j) \mid R_{uj} = 0\}$, t_+, t_- (again, a notation abuse) are node degree-based coefficients calculated on the user-item graph, and Sigmoid$(\cdot)$ is the sigmoid function. Regarding the item loss, it is calculated by considering the item-projected graph, thus it is formalized as follows:

$$\mathcal{L}_{\text{IL}} = - \sum_{(u,i) \sim \mathcal{T}_+} \sum_{j \in \text{topK}(\mathcal{T}_+^{\mathcal{II}})} t_-^{\mathcal{II}} \ln(\text{Sigmoid}(\hat{\mathbf{E}}_u^\top \hat{\mathbf{E}}_j)), \tag{3.20}$$

where topK is the function returning the top K values of a set, $\mathcal{T}_+^{\mathcal{II}} = \{(i,j) \mid A_{ij}^{\mathcal{II}} \neq 0\}$, and $t_-^{\mathcal{II}}$ is a node degree-based coefficient calculated on the item-item graph.

3.5.6 GFCF

Graph filter-based collaborative filtering (Shen *et al.*, 2021) (GFCF) questions the role of graph convolutional network into recommendation by leveraging graph signal processing theory. Indeed, the authors show

that several existing approaches in CF may fall into one unified framework based upon graph convolution, to eventually propose a closed-form algorithm that proves to be a strong baseline against other trainable and computationally-expensive (graph-based) approaches in CF.

$$\hat{\mathbf{R}}_{u*} = \mathbf{R}_{u*} \left(\left(\tilde{\mathbf{A}}^{\mathcal{UI}} \right)^{\top} \tilde{\mathbf{A}}^{\mathcal{IU}} + t \left(\mathbf{D}^{\mathcal{II}} \right)^{-\frac{1}{2}} \text{topK}(\mathbf{U}) \text{topK}(\mathbf{U})^{\top} \left(\mathbf{D}^{\mathcal{II}} \right)^{+\frac{1}{2}} \right), \quad (3.21)$$

where we are re-using the notation we introduced in Equation 3.2 and Equation 3.5 to indicate different partitions of the (normalized) adjacency matrix and degree matrix, while t is a tunable coefficient, and $\mathbf{U}$ is the vector collecting the singular vectors of $\tilde{\mathbf{A}}^{\mathcal{UI}}$.

3.5.7 SVD-GCN

Similarly to what was observed with GFCF, the model proposed by (Peng *et al.*, 2022b) leverages the conceptual similarities between graph convolution and singular value decomposition (SVD) to design a novel approach which also explores user-user and item-item co-occurrences. Concretely, the authors propose a light-weight trainable recommender model, where user and item embeddings are obtained as:

$$\hat{\mathbf{E}}_u = (\mathbf{U}) \exp(t\mathbf{V}) \mathbf{W}, \quad (3.22)$$

where $\mathbf{U}$ is the vector collecting the singular vectors of $\tilde{\mathbf{A}}^{\mathcal{UI}}$ (as in GFCF), t is a tunable parameter, $\mathbf{V}$ is vector of the largest singular values on $\tilde{\mathbf{A}}^{\mathcal{UI}}$, and $\mathbf{W}$ is a trainable matrix. Finally, the model optimizes the following user (item) loss function (i.e., UL/IL) on the user-user (item-item) co-occurrences graph (here, we only report the user side):

$$\mathcal{L}_{\text{UL}} = \sum_{u \in \mathcal{U}} \sum_{(u,v) \sim \mathcal{T}_{+}^{\mathcal{UU}},\, (u,w) \sim \mathcal{T}_{-}^{\mathcal{UU}}} \ln \text{Sigmoid}(\hat{\mathbf{E}}_u^{\top} \hat{\mathbf{E}}_v - \hat{\mathbf{E}}_u^{\top} \hat{\mathbf{E}}_w), \quad (3.23)$$

where $\mathcal{T}_{+}^{\mathcal{UU}} = \{(u, v) \mid A_{uv}^{\mathcal{UU}} \neq 0\}$ and $\mathcal{T}_{-}^{\mathcal{UU}} = \{(u, w) \mid A_{uw}^{\mathcal{UU}} = 0\}$.

3.5.8 SimGCL

The authors in (Yu *et al.*, 2022) empirically observe that graph augmentations, such as those proposed in (Wu *et al.*, 2021), are not the actual reason behind the improved recommendation performance, but rather

uniformity of the node embeddings' distribution. Thus, they propose SimGCL, short for Simplification of Graph Contrastive Learning, a novel approach that simplifies graph contrastive learning by regulating the uniformity. Concretely, they propose to perturb the node embeddings during the message passing procedure in this way:

$$\begin{aligned}\mathbf{E}_u^{(l)}[t_1] &= \sum_{i \in \mathcal{N}_u} \tilde{A}_{ui} \mathbf{E}_i^{(l-1)}[t_1] + \Delta^{(l)}[t_1] \\ \mathbf{E}_u^{(l)}[t_2] &= \sum_{i \in \mathcal{N}_u} \tilde{A}_{ui} \mathbf{E}_i^{(l-1)}[t_2] + \Delta^{(l)}[t_2],\end{aligned} \tag{3.24}$$

where $||\Delta^{(l)}[t_*]||_2 = \epsilon$, $\Delta^{(l)}[t_*] = \bar{\Delta}^{(l)}[t_*] \odot \text{sign}(\mathbf{E}_i^{(l-1)})$, and $\bar{\Delta}^{(l)}[t_*] \sim \text{uniform}(0, 1)$. Finally, the model optimizes a composite loss with BPR and a CL (as in SGL), along with a uniform loss (UNL) to ensure the uniformity of the learned representations:

$$\mathcal{L}_{\text{UNL}} = \ln \left(\mathop{\mathbb{E}}_{u,v} \left[\exp \left(-2 \left\| \frac{\mathbf{E}_u}{||\mathbf{E}_u||_2} - \frac{\mathbf{E}_v}{||\mathbf{E}_v||_2} \right\|_2^2 \right) \right] \right), \tag{3.25}$$

where u, v are iid from the set of most popular items in the catalog.

3.5.9 LightGCL

The authors in (Cai *et al.*, 2023) propose a novel approach named LightGCL, a simple yet effective graph contrastive learning solution. The framework consists of a double message passing procedure acting in the local and global view of the user-item interaction graph. As for the local view, the model adopts the same propagation layer proposed in (Xia *et al.*, 2022b), namely:

$$\mathbf{E}_u^{(l)} = \text{LeakyReLU} \left(\text{drop}(\tilde{\mathbf{A}}_{u*}) \mathbf{E}_i^{(l-1)} \right) + \mathbf{E}_i^{(l-1)}, \tag{3.26}$$

where drop() corresponds to the edge dropout function to avoid overfitting, and a residual connection with the previous representation of the node embedding is adopted. Conversely, at global view, the authors propose to approximate the adjacency matrix through its SVD, and in particular the approximated SVD where top K singular values are retained:

$$\mathbf{G}_u^{(l)} = \text{LeakyReLU} \left(\hat{\mathbf{A}}_{u*} \mathbf{E}_i^{(l-1)} \right) = \text{LeakyReLU} \left(\hat{\mathbf{U}}_K \hat{\mathbf{S}}_K \hat{\mathbf{V}}_K^\top \mathbf{E}_i^{(l-1)} \right), \tag{3.27}$$

where $\hat{\mathbf{U}}_K \in \mathbb{R}^{U \times K}$ and $\hat{\mathbf{V}}_K \in \mathbb{R}^{I \times K}$ are the eigenvectors of the adjacency matrix when considering the top K largest columns of both vectors, and $\hat{\mathbf{S}}_K \in \mathbb{R}^{K \times K}$ is the diagonal matrix of the top K largest eigenvalues. On such a basis, a contrastive loss is defined between the local and global views of the same node embeddings, obtaining:

$$\mathcal{L}_{\text{CL}} = \sum_{u \in \mathcal{U}} \sum_{l \in \{0,\dots,L\}} -\ln \frac{\exp(\text{Sim}(\mathbf{E}_u^{(l)}, \mathbf{G}_u^{(l)})/t)}{\sum_{v \in \mathcal{U} \setminus \{u\}} \exp(\text{Sim}(\mathbf{E}_u^{(l)}, \mathbf{G}_v^{(l)})/t)}, \tag{3.28}$$

where $\mathbf{E}_*^{(l)}$ and $\mathbf{G}_*^{(l)}$ refer to the local and global views computed at layer l, respectively, while t is the common temperature coefficient to rescale the similarity function $\text{Sim}(\cdot)$. Note that, in the above equation, the model performs node dropout to select specific nodes for the calculation to prevent overfitting. Alongside the contrastive loss, the model pursues the recommendation task by optimizing the hinge pairwise loss:

$$\mathcal{L}_{\text{HL}} = \sum_{u \in \mathcal{U}} \sum_{(u,i,j) \in \mathcal{T}} \max\left(0, 1 - \hat{\mathbf{E}}_u^\top \hat{\mathbf{E}}_i\right). \tag{3.29}$$

3.5.10 GraphAU

The authors in (Yang *et al.*, 2023) leverage the rationale from (Wang *et al.*, 2022a) to adapt the uniformity and alignment paradigm for graph-based recommendation within their GraphAU framework. First, they refine the node embeddings for users and items through the LightGCN message passing layer. On such a basis, the overall loss is calculated as a trade-off between the alignment (AL) and uniformity (UNL) losses, where the former is specifically tailored in this work for graph learning:

$$\mathcal{L} = \frac{1}{|\mathcal{T}|} \sum_{(u,i) \in \mathcal{T}} \mathcal{L}_{\text{AL}} + \frac{t_1}{2} \mathcal{L}_{\text{UNL}}, \tag{3.30}$$

where the AL is calculated as:

$$\mathcal{L}_{\text{AL}} = \sum_{l \in \{0,\dots,L\}} \frac{(t_2)^l}{2} \left(||\mathbf{E}_u^{(0)} - \mathbf{E}_i^{(l)}||^2 + ||\mathbf{E}_i^{(0)} - \mathbf{E}_u^{(l)}||^2 \right). \tag{3.31}$$

Note that, in the above equations, t_1 and t_2 correspond to the trade-off coefficient between the AL and the UNL and the importance weight of the AL at each propagation layer, respectively.

3.5.11 XSimGCL

Unlike previous contrastive learning approaches (such as SimGCL), the model proposed in (Yu *et al.*, 2024), named XSimGCL (short for extremely simple graph contrastive learning), suggests using the same contrasted view of the node embeddings also for the recommendation task. In addition to this, the authors adopt a cross-layer contrastive module which calculates the InfoNCE between the same node view calculated at different propagation layers. First, the model computes the augmented version of the embedding at layer l as in SimGCL, but only for one view t:

$$\mathbf{E}_u^{(l)}[t] = \sum_{i \in \mathcal{N}_u} \tilde{A}_{ui} \mathbf{E}_i^{(l-1)}[t] + \Delta^{(l)}[t]. \tag{3.32}$$

Then, after L propagation layers, the loss is calculated as the combination of BPR and the InfoNCE where the contrasting views are obtained from the same augmented node embedding but at layers L and at a specific layer $l^* < L$:

$$\begin{aligned} \mathcal{L}_{\text{BPR}} &= - \sum_{(u,i,j) \in \mathcal{T}} \ln \text{Sigmoid}(\hat{\mathbf{E}}_u[t]^\top \hat{\mathbf{E}}_i[t] - \hat{\mathbf{E}}_u[t]^\top \hat{\mathbf{E}}_j[t]), \\ \mathcal{L}_{\text{CL}} &= \sum_{i \in \mathcal{I}} -\ln \frac{\exp(\text{Sim}(\hat{\mathbf{E}}_i, \mathbf{E}_i^{(l^*)}))}{\sum_{j \in \mathcal{I} \setminus \{i\}} \exp(\text{Sim}(\hat{\mathbf{E}}_i, \mathbf{E}_j^{(l^*)}))}. \end{aligned} \tag{3.33}$$